Lecture 7

15 October 2026 23:28

Moving towards the GPT

1.C.1

1. Removing the vocab size in the __init__ constructor -----> only the self
2. The vocab size in the embedding table is remove and change to embedding dimension

```
def __init__(self):
    super().__init__()
    # each token directly reads off the logits for the next token from a lookup table
    self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
```

In the Bigram model, embedding_dim = vocab_size = 65 was used as a **shortcut**. The 65 numbers in each row served DIRECTLY as logits (one score per character in vocabulary).

But this has a big problem:

The embedding is being forced to do two completely different jobs at the same time!

Job	What it means
Job 1: Represent the token	Give each character a rich feature vector for learning relationships
Job 2: Predict next token	Output 65 scores (one per vocabulary character)

These two jobs should be **separated**!

1 : What n_embd does:

n_embd (number of embedding dimensions) is a **hyperparameter** you choose freely (e.g., n_embd = 32):

python
n_embd = 32

1. # Embedding table: purely for REPRESENTING tokens richly
self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
Shape: (65, 32) — 65 tokens, each with 32 rich features

2. # NEW: A separate Linear layer purely for PREDICTING next token
self.lm_head = nn.Linear(n_embd, vocab_size)
Takes 32 features → outputs 65 logits (one per character)

2. The New Forward Pass:

python
def forward(self, idx, targets=None):

```
    # Step 1: Lookup rich 32-dimensional embedding for each token
    tok_emb = self.token_embedding_table(idx)   # (B, T, n_embd) = (B, T, 32)

    # Step 2: Pass through Self-Attention layers (coming soon!)
    # x = self_attention(tok_emb)

    # Step 3: Project 32 features → 65 vocabulary logits
    logits = self.lm_head(tok_emb)              # (B, T, vocab_size) = (B, T, 65)
```

Visual comparison:

text
OLD (Bigram):
Token ID → [Embedding Table 65x65] → 65 logits directly

NEW (GPT-style):
Token ID → [Embedding Table 65x32] → 32 rich features → [Linear Layer 32x65] → 65 logits

Why is this better?

1. **Richer Representations:** With n_embd = 32 (or 384 in nanoGPT), each token gets a much richer internal representation that can capture complex patterns.
2. **Separation of Concerns:** The embedding layer focuses purely on learning what each token means. The lm_head Linear layer focuses purely on converting that meaning into vocabulary predictions.

3. **Scalability:** You can now increase n_embd independently of vocab_size. GPT-3 uses n_embd = 12,288 while vocab_size stays at 50,257.

What you got right :

- The embedding now outputs n_embd features (32) instead of 65.
- lm_head then converts those features back into 65 logits for prediction.
- Self-Attention helps enrich the small feature vectors with context before predicting.

The REAL reason for separating n_embd from vocab_size:

The old Bigram model forced the embedding to do **two jobs at once** in one single matrix:

text
OLD (65 × 65):
Job 1: "Represent what 'h' means" ← embedding job
Job 2: "Score all 65 next characters" ← prediction job
Both crammed into ONE matrix!

The new design **separates** these two jobs into two dedicated components:

text
NEW:
[Embedding Table 65 × 32]: ONLY Job 1 — "Represent what 'h' means richly"
↓
[Self-Attention]: "Mix context from past tokens to enrich representation"
↓
[lm_head 32 → 65]: ONLY Job 2 — "Convert rich representation → 65 logits"

Clean Summary for your notes:

	Old Bigram	New GPT-style
Embedding size	65 × 65 (forced equal)	65 × n_embd (freely chosen)
Embedding job	Represent token AND predict next	Only represent token richly
Prediction job	Done by embedding directly	Done by separate lm_head
Self-Attention	Not present	Enriches features between embedding and lm_head

The key insight is not "less information" — it is **separation of responsibilities** + **freedom to choose any n_embd size** that makes the model most powerful!

The changes,

1. This embedding table only for lookup, and creating 32 rich features

```
# Embedding table: purely for REPRESENTING tokens richly
self.token_embedding_table = nn.Embedding(vocab_size, n_embd)  # Shape: (65, 32) — 65 tokens, each with 32 rich features
```

2. Creating new token i.e. predicting new tokens

```
# NEW: A separate Linear layer purely for PREDICTING next token
self.lm_head = nn.Linear(n_embd, vocab_size)  # Takes 32 features → outputs 65 logits (one per character)
```

3. From the look up, we will get token embedding, i.e. only the features for each batch row

```
# Step 1: Lookup rich 32-dimensional embedding for each token
tok_emb = self.token_embedding_table(idx)  # (B, T, n_embd) = (B, T, 32), tok_emb = token will be represented from the embedding table
```

4. And we will make new logits

```
# Step 3: Project 32 features → 65 vocabulary logits
logits = self.lm_head(tok_emb)  # (B, T, vocab_size) = (B, T, 65), logits are generated for each character
```

5. self.position_embedding_table = nn.Embedding(block_size, n_embd)

This is **Positional Embedding** — the second embedding table Karpathy adds!

The Problem: Self-Attention has no sense of order!

When we do wei @ x in Self-Attention, the operation treats tokens like an **unordered bag**. It has no idea whether 'h' came at position 0 or position 5!

text
"hello" → ['h', 'e', 'l', 'l', 'o']
"ohlel" → ['o', 'h', 'l', 'e', 'l']
To Self-Attention, these two sentences look identical because it only sees WHAT the tokens are, not WHERE they are

The Solution: Positional Embedding Table

python
```
self.position_embedding_table = nn.Embedding(block_size, n_embd)
#                                8 rows,     32 columns
```

This creates a **second lookup table** where:

- **Rows** = each possible position in the sequence (0, 1, 2, 3, 4, 5, 6, 7)
- **Columns** = n_embd learned features describing that position

Position Embedding Table (block_size × n_embd) = (8 × 32):
Position 0: [0.11, -0.43, 0.72, ...] ← "I am the 1st position"
Position 1: [0.55, 0.21, -0.33, ...] ← "I am the 2nd position"
Position 2: [-0.12, 0.88, 0.41, ...] ← "I am the 3rd position"
...
Position 7: [0.33, -0.71, 0.19, ...] ← "I am the 8th position"

6. pos_emb = self.position_embedding_table(torch.arange(T, device=device))

This helps to arrange the words in order, like, the hello and gives integer numbers to them 0, 1, 2, like that.....

Changes Part: 1

What we have done so far,.....

1. self.token_embedding_table = nn.Embedding(vocab_size, n_embd) ---> step 1 : in __init__ function in the BigramLanguageModel, for token ID lookup
2. self.lm_head = nn.Linear(n_embd, vocab_size) ---> step 2 : predicting the future token

Exp

1. tok_emb = self.token_embedding_table(idx) ---> step 1 : in forward pass, to take a row from the embedding table
2. logits = self.lm_head(tok_emb) ---> step 2 : In forward pass, to store that predicted token as a logits

Part: 2

1. self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
2. self.position_embedding_table = nn.Embedding(block_size, n_embd)
3. self.lm_head = nn.Linear(n_embd, vocab_size)

V IMP:
We have created one more embedding table for the self-attention, as the self-attention block does not look where the position of the featured token is like

It see both this as same :
'h' 'e' 'l' 'l' 'o' and
'e' 'h' 'l' 'o' 'l'

And in the forward loop:

1. tok_emb = self.token_embedding_table(idx)
2. Pos_emb = self.position_embedding_table(torch.arange(T, device=device))

   V IMP :
3. x = tok_emb + pos_emb

4. logits = self.lm_head(x)

Why ADD them instead of concatenate?

Adding tok_emb + pos_emb combines two pieces of information into the same n_embd-dimensional space:

text
tok_emb[0, 2]: [0.44, 1.21, -0.88, ...] ← "I am the letter 'l'"
pos_emb[2]: [-0.12, 0.88, 0.41, ...] ← "I am at position 2"
x[0, 2]: [0.32, 2.09, -0.47, ...] ← "I am 'l' at position 2"

The resulting vector carries **both identities at once**!

Summary for your notes:

Embedding	Table Size	Lookup Key	What it encodes
token_embedding_table	vocab_size × n_embd	Token ID (e.g., 12 for 'h')	**WHAT** the token is
position_embedding_table	block_size × n_embd	Position index (0, 1, 2...)	**WHERE** the token is
Combined (x)	(B, T, n_embd)	Both added together	**WHAT + WHERE**

Without positional embedding, "hello" and "ohlel" would look identical to Self-Attention. With it, every token knows both its identity AND its position in the sequence!